



자율주행자동차의 딜레마 상황에서의 윤리적 판단 문제

- 문제분석: 최근 과학의 발달로 인간이 아닌 자율주행자동차가 스스로 조종할 수 있게 됨 -> 윤리적 판단 문제상황을 만났을 때 어떠한 판단을 해야하는지에 대한 판단기준을 세움

- 벤담의 기준에 따라 우리가 정한 점수의 책정기준 “공리주의”

1. 행복이나 쾌락의 총합이 더 높은 개념에 더 높은 점수 부여
2. 벤담의 공리개념을 보다 구체적으로 적용하기 위해 행복 계산법으로 가중치를 두어 특정값을 대입함

강도 - 쾌락이나 고통의 강도 -> 강한 쾌락- 생존 : -2
약한 고통- 부상: -5
강한 고통- 사망 : -10

지속성 - 벤담의 공리주의 관점에서 어떤 행위가 강도가 높은 쾌락을 제공하더라도 그 지속성이 짧다면, 지속성이 길지만 강도가 낮은 쾌락과 비교하여 총 쾌락의 양에서 밀릴 수 있음

-> 연령대가 젊을수록 쾌락점수 높게 부여
유아: 10 , 청소년 : 8, 청년: 6, 중년: 4 , 노인: 2

다산성- 단기적 결과 뿐만 아니라 장기적 결과를 고려

ex) 직업 : 타인에게 **쾌락**을 제공하는 직업일수록 점수를 높게 부여

1. 의료 및 치유 분야

-심리 치료사/상담사/의사 : 10

2. 교육 및 학습 분야

-교사/강사: 8

3. 엔터테인먼트 및 예술 분야

-배우/연예인: 5

4. 프로그래머 : 4

5. 백수: 3

6. 범죄자: 2

범위- “최대 다수의 최대 행복” 추구

어떤 행동이 더 많은 사람에게 긍정적인 영향을 미칠수록
그 행동은 도덕적으로 가중치를 둠

-> 사람 수가 더 많은 쪽을 선택할수록 점수가 적음

EX) 5명: -5, 4명: -4, 3명: -3

순수성- 생존이라는 쾌락을 제공할 때, 그 쾌락이 순수하게 쾌락만을 제공해야 함

경제적 계층에 따른 점수 부여

상류층: 9

중산층: 5

하류층: 2

근접성- 벤담의 공리주의 에서 쾌락이나 고통이 발생하는 시간적 거리

범위, 강도 : 즉각적인 반응 (-)

다산성, 지속성, 확실성, 불확실성: 어느정도 시간이 흐른 후 쾌락을 제공하거나 느낌
(+)

가중치 계산 방법- 강도(2) < 범위(3) < 순수성(4) < 지속성(5) < 다산성(6)

차선 변경 시의 딜레마

고속도로에서 자율주행 자동차가 앞서 가는 차량이 갑자기 정차한 것을 발견합니다. 한쪽 차선에는 여러 명의 작업자가 도로에서 작업 중이고, 다른 쪽 차선에는 한 명의 작업자가 있습니다.

선택지 1. 3명의 사람을 보호하기 위해 4 명의 작업자와 충돌한다.

선택지 2. 4명의 작업자를 보호하기 위해 3의 작업자와 충돌한다.

선택지 3. 차를 도로 밖으로 몰아 자차 탑승자 5명에게 피해를 준다.

```
import numpy as np
import tkinter as tk
from tkinter import simpledialog, messagebox

weights = {
    "fatalities": 3, # 사망자의 수에 큰 가중치를 부여
    "lifeloss": 5,    # 부상자의 수에 중간 가중치를 부여
    "moneygain": 4,   # 재산 손실에 작은 가중치를 부여
    "liveornot": 2,
    "jobgain": 6
}
```



```

def scenario1():
    lifeloss1 = 0
    for i in range(s1):
        root = tk.Tk()
        root.withdraw() # 메인 창 숨기기
        choices = ["유아", "청소년", "청년", "중년", "노인"]
        choice = simpledialog.askstring(title="Input", "시나리오 1에서 사람들의 연령대를 선택하세요:\n" + "\n".join(choices))
        messagebox.showinfo(title="Result", message=f"You selected: {choice}")
        if choice == "유아" :
            lifeloss1 -= 10
        elif choice == "청소년":
            lifeloss1 -= 8
        elif choice == "청년":
            lifeloss1 -= 6
        elif choice == "중년":
            lifeloss1 -= 4
        elif choice == "노인":
            lifeloss1 -= 2
    return lifeloss1
total_lifeloss1 = scenario1()
print("Total lifeloss:", total_lifeloss1)

! usage
def scenario2():
    lifeloss2 = 0

    for i in range(s2):
        root = tk.Tk()
        root.withdraw() # 메인 창 숨기기
        choices = ["유아", "청소년", "청년", "중년", "노인"]
        choice = simpledialog.askstring(title="Input", "시나리오 1에서 사람들의 연령대를 선택하세요:\n" + "\n".join(choices))
        messagebox.showinfo(title="Result", message=f"You selected: {choice}")
        if choice == "유아" :
            lifeloss2 -= 100
            choices = ["부자", "중산층", '가난함']
        elif choice == "청소년":
            lifeloss2 -= 80
        elif choice == "청년":
            lifeloss2 -= 60
        elif choice == "중년":

```

```

def scenario3():
    lifeloss3 = 0
    for i in range(s2):
        root = tk.Tk()
        root.withdraw() # 메인 창 숨기기
        choices = ["유아", "청소년", "청년", "중년", "노인"]
        choice = simpledialog.askstring(title="Input", "시나리오 1에서 사람들의 연령대를 선택하세요:\n" + "\n".join(choices))
        messagebox.showinfo(title="Result", message=f"You selected: {choice}")
        if choice == "유아" :
            lifeloss3 -= 100
        elif choice == "청소년":
            lifeloss3 -= 80
        elif choice == "청년":
            lifeloss3 -= 60
        elif choice == "중년":
            lifeloss3 -= 40
        elif choice == "노인":
            lifeloss3 -= 20
    return lifeloss3

```

```
s1 = int(input('1선택 사람 수 : '))
s2 = int(input('2선택 사람 수 : '))
s3=  int(input('3선택 사람 수 : '))
```

3 usages

```
def moneycal(kk):
```

```
    moneygain = 0
```

```
    for i in range(kk):
```

```
        root = tk.Tk()
```

```
        root.withdraw() # 메인 창 숨기기
```

```
        choices = ["상류층", "중산층", "하류층"]
```

```
        choice = simpledialog.askstring( title: "Input", "시나리오에서 사람들의 경제적 상황을 선택하세요:\n" + "\n".join(choices))
```



```
        messagebox.showinfo( title: "Result", message: f"You selected: {choice}")
```

```
        if choice == "상류층":
```

```
            moneygain += 9
```

```
        elif choice == "중산층":
```

```
            moneygain += 5
```

```
        elif choice == "하류층":
```

```
            moneygain += 2
```

```
    return moneygain
```

```

s1 = int(input('1선택 사람 수 : '))
s2 = int(input('2선택 사람 수 : '))
s3= int(input('3선택 사람 수 : '))
3 usages
def moneycal(kk):
    moneygain = 0
    for i in range(kk):
        root = tk.Tk()
        root.withdraw() # 메인 창 숨기기
        choices = ["상류층", "중산층", "하류층"]
        choice = simpledialog.askstring(title: "Input", "시나리오에서 사람들의 경제적 상황을 선택하세요:\n" + "\n".join(choices))
        messagebox.showinfo(title: "Result", message: f"You selected: {choice}")
        if choice == "상류층":
            moneygain += 9
        elif choice == "중산층":
            moneygain += 5
        elif choice == "하류층":
            moneygain += 2
    return moneygain

3 usages
def liveornot(kk):
    iflife = 0
    for i in range(kk):
        root = tk.Tk()
        root.withdraw() # 메인 창 숨기기
        choices = ["사망", "부상", "생존"]
        choice = simpledialog.askstring(title: "Input", "시나리오에서 사람들의 생존 유무를 설정하세요:\n" + "\n".join(choices))
        messagebox.showinfo(title: "Result", message: f"You selected: {choice}")
        if choice == "사망":
            iflife -= 10
        elif choice == "부상":
            iflife -= 5
        elif choice == "생존":
            iflife -= 2
    return iflife

```

```

3 usages
def jobcal(kk):
    jobgain=0
    for i in range(kk):
        root = tk.Tk()
        root.withdraw() # 메인 창 숨기기
        choices = ["의사", "교사", "범죄자", "백수", "프로그래머"]
        choice = simpledialog.askstring(title: "Input", "시나리오에서 사람들의 직업을 선택하세요:\n" + "\n".join(choices))
        messagebox.showinfo(title: "Result", message: f"You selected: {choice}")
        if choice == "의사":
            jobgain += 10
        elif choice == "교사":
            jobgain += 8
        if choice == "범죄자":
            jobgain += 10
        elif choice == "백수":
            jobgain += 3
        elif choice == "프로그래머":
            jobgain += 4
    return jobgain

3 usages
def lifecal(kk):
    lifeloss = 0
    for i in range(kk):
        root = tk.Tk()
        root.withdraw() # 메인 창 숨기기
        choices = ["유아", "청소년", "청년", "중년", "노인"]
        choice = simpledialog.askstring(title: "Input", "시나리오에서 사람들의 연령대를 선택하세요:\n" + "\n".join(choices))
        messagebox.showinfo(title: "Result", message: f"You selected: {choice}")
        if choice == "유아":
            lifeloss -= 10
        elif choice == "청소년":
            lifeloss -= 8
        elif choice == "청년":
            lifeloss -= 6
        elif choice == "중년":
            lifeloss -= 4
        elif choice == "노인":
            lifeloss -= 2

```

```

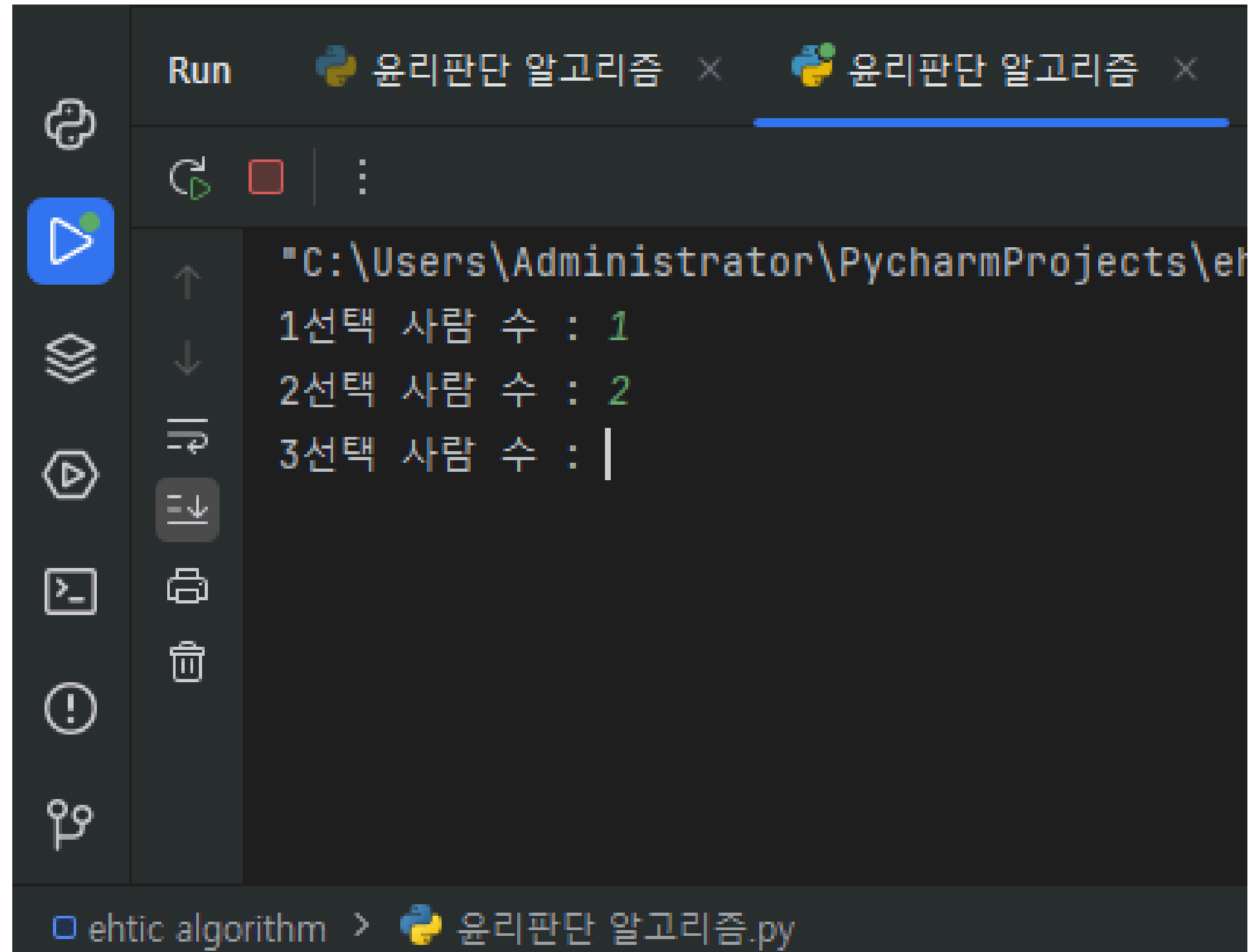
89     return lifeloss
90     print("제 1 시나리오")
91     kk=s1
92     total_lifeloss1 = lifecal(kk)
93     print("lifeloss1 :", total_lifeloss1)
94     total_moneygain1 =moneycal(kk)
95     print("moneygain1 : ", total_moneygain1)
96     total_jobgain1 =jobcal(kk)
97     print("jobgain1 : ", total_moneygain1)
98     total_iflife1 =liveornot(kk)
99     print("iflife1 : ", total_iflife1)
100
101     print("제 2 시나리오")
102     kk=s2
103     total_lifeloss2 = lifecal(kk)
104     print("lifeloss2 :", total_lifeloss2)
105     total_moneygain2 =moneycal(kk)
106     print("moneygain2 : ", total_moneygain2)
107     total_jobgain2 =jobcal(kk)
108     print("jobgain2 : ", total_jobgain2)
109     total_iflife2 =liveornot(kk)
110     print("iflife2 : ", total_iflife2)
111
112     print("제 3 시나리오")
113     kk=s3
114     total_lifeloss3 = lifecal(kk)
115     print("lifeloss3 :", total_lifeloss3)
116     total_moneygain3 =moneycal(kk)
117     print("moneygain3 : ", total_moneygain3)
118     total_jobgain3 =jobcal(kk)
119     print("jobgain3 : ", total_jobgain3)
120     total_iflife3 =liveornot(kk)
121     print("iflife3 : ", total_iflife3)

```

```

( (col11011 + col101010) = 0110101 , (col111001 + 0 , col110000 + col101 = col110000)

```

PC

EA ehtic algorithm Version control

Current File

Project

ehthic algorithm C:\Users\Administrator\PycharmProjects\ehthic algorithm

.env library root

Lib

Scripts

.gitignore

pyvenv.cfg

윤리판단 알고리즘.py

External Libraries

Scratches and Consoles

윤리판단 알고리즘.py

```
71 def lifecal(kk):
83     elif choice == "청년":
84         lifeloss -= 6
85     elif choice == "중년":
86         lifeloss -= 4
87     elif choice == "노인":
88         lifeloss -= 2
89     return lifeloss
90 print("제 1 시나리오")
91 kk=s1
92 total_lifeloss1 = lifecal(kk)
93 print("lifeloss1 :", total_lifeloss1)
94 total_moneygain1 = moneycal(kk)
95 print("moneygain1 :", total_moneygain1)
96 total_jobgain1 = jobcal(kk)
97 print("jobgain1 :", total_moneygain1)
98 total_iflife1 = liveornot(kk)
99 print("iflife1 :", total_iflife1)
100
101 print("제 2 시나리오")
102 kk=s2
103 total_lifeloss2 = lifecal(kk)
104 print("lifeloss2 :", total_lifeloss2)
105 total_moneygain2 = moneycal(kk)
106 print("moneygain2 :", total_moneygain2)
107 total_jobgain2 = jobcal(kk)
108 print("jobgain2 :", total_jobgain2)
109 total_iflife2 = liveornot(kk)
110 print("iflife2 :", total_iflife2)
```

Run 윤리판단 알고리즘 윤리판단 알고리즘

1선택 사람 수 : 1

2선택 사람 수 : 2

3선택 사람 수 : 3

제 1 시나리오

lifeloss1 : -10

moneygain1 : 9

jobgain1 : 9

iflife1 : -2

제 2 시나리오

ehthic algorithm 윤리판단 알고리즘.py

146:32 CRLF UTF-8 4 spaces Python 3.12 (ehthic algorithm)

Windows Taskbar

Input

시나리오에서 사람들의 연령대를 선택하세요:

유아

청소년

청년

중년

노인

OK Cancel

Input

시나리오에서 사람들의 연령대를 선택하세요:

유아

청소년

청년

중년

노인

OK Cancel



```
"C:\Users\Administrator\PycharmProjects\ehitic algorithm\.venv\Scripts\python.exe" "C:\U
```

```
1선택 사람 수 : 1
```

```
2선택 사람 수 : 2
```

```
3선택 사람 수 : 3
```

```
제 1 시나리오
```

```
lifeloss1 : -10
```

```
moneygain1 : 9
```

```
jobgain1 : 9
```

```
iflife1 : -2
```

```
제 2 시나리오
```

```
lifeloss2 : -20
```

```
moneygain2 : 11
```

```
jobgain2 : 7
```

```
iflife2 : -12
```

```
제 3 시나리오
```

```
lifeloss3 : -8
```

```
moneygain3 : 16
```

```
jobgain3 : 21
```

```
iflife3 : -17
```

```
시나리오 분석 결과 :
```

```
최선의 결정: do_nothing, 쾌락고통 점수: -47
```

```
최선의 결정: turn_left, 쾌락고통 점수: -94
```

```
최선의 결정: turn_right, 쾌락고통 점수: -31
```

```
최상의 시나리오: turn_right
```